Lab 13 — Praktijklab: de keten web → api in het cluster

Doel: de frontend en de API uitrollen, nginx zien sterven bij gebrek aan Service, het Service `api` aanmaken en DNS, cluster-IP en EndpointSlices observeren, de frontend blootstellen als NodePort en dan als LoadBalancer, en ten slotte via de Traefik-Ingress op poort 80 — en nagaan dat de endpoints de Pods volgen.

Vereisten — Lab 12 afgewerkt: k3s, `localhost/api:1.0` in het cluster, Podman met `api:1.0`.

Meegeleverde bestanden (`files/`) - `Dockerfile.web` , `nginx-proxy.conf` , `index.html` — de image `web:1.0` : nginx + de frontend + de proxy `/api/` → `http://api:8080/` . - `deployment-api.yaml` , `deployment-web.yaml` — de twee Deployments (dat van de API is dat van lab 12). - `service-api.yaml` (ClusterIP), `service-web.yaml` (NodePort 30080), `ingress-web.yaml` (Traefik, pad `/`).

Stap 0 — De image van de frontend in het cluster

```
mkdir -p ~/labo-docker/13 && cd ~/labo-docker/13 && cp <pad-naar-het-lab>/files/* .
podman build -q -f Dockerfile.web -t web:1.0 .
podman save localhost/web:1.0 | sudo k3s ctr images import -
sudo k3s crictl images | grep -E 'localhost/(api|web)'
```

Observeer `localhost/web 1.0` naast de `localhost/api` -images in het cluster.

Stap 1 — Zonder Service, geen naam

```
kubectl create namespace lab13 ; kubectl config set-context --current --namespace=lab13
kubectl apply -f deployment-api.yaml -f deployment-web.yaml
kubectl rollout status deployment/api --timeout=60s
sleep 20 ; kubectl get pods
kubectl logs deploy/web --tail=1
```

Observeer de API `Running` en `web` in `CrashLoopBackOff` met `RESTARTS 2` , en de oorzaak: `nginx: [emerg] host not found in upstream "api" in /etc/nginx/conf.d/default.conf:15` .

Uitleg. In lab 07 benoemde het Podman-netwerk elke container. Hier heeft een Pod geen DNS-naam: nginx resolvet `api` bij het opstarten, vindt niets, en eindigt. Alleen een Service draagt een naam.

Stap 2 — Het ClusterIP-Service

```
kubectl apply -f service-api.yaml
kubectl rollout status deployment/web --timeout=120s
kubectl get pods
kubectl get svc api ; kubectl get endpointslices
```

Observeer `web` dat `1/1 Running` geworden is **zonder enig commando van jou** (`RESTARTS 3`, de backoff is uiteindelijk geslaagd); een Service `api ClusterIP 10.43.x.x <none> 8080/TCP`; en een EndpointSlice `api-xxxxx` met de twee IP's van de API-Pods.

```
kubectl exec deploy/web -- wget -qO- http://api:8080/actuator/health ; echo
kubectl exec deploy/web -- cat /etc/resolv.conf
kubectl exec deploy/web -- wget -qO- http://api.lab13.svc.cluster.local:8080/ ; echo
kubectl scale deployment api --replicas=3 ; sleep 8 ; kubectl get endpointslices
```

Observeer `{"status":"UP"}` vanuit de `web`-Pod; een `resolv.conf` met `search lab13.svc.cluster.local svc.cluster.local cluster.local` en `nameserver 10.43.0.10`; hetzelfde antwoord via de volledige naam; en, na de overgang naar drie replica's, **drie** adressen in de EndpointSlice.

Uitleg. Het Service heeft een virtueel IP en een naam; CoreDNS resolvet de naam, kube-proxy vertaalt het IP naar een van de endpoints, en de lijst endpoints volgt de klare Pods.

Stap 3 — NodePort: een deur op de node

```
kubectl apply -f service-web.yaml ; sleep 3 ; kubectl get svc web
NODE=$(kubectl get nodes -o jsonpath='{.items[0].status.addresses[0].address}') ; echo $NODE
curl -s http://$NODE:30080/ | head -3
curl -s http://$NODE:30080/api/actuator/health ; echo
```

Observeer `web NodePort 10.43.x.x <none> 80:30080/TCP`, het IP van je node (dat van de WSL-distributie), de `<!doctype html>` van de frontend en `{"status":"UP"}` via de nginx-proxy — alles op poort `30080` van de node. Vanuit Windows opent `http://localhost:30080/` dezelfde pagina.

Uitleg. Een NodePort opent dezelfde poort op elke node en geeft ze door naar het Service. Handig om te testen; in productie verkies je een adres zonder exotische poort.

Stap 4 — LoadBalancer: ServiceLB van k3s

```
kubectl expose deployment web --name=web-lb --type=LoadBalancer --port=8081 --target-port=80
sleep 10 ; kubectl get svc web-lb
kubectl get pods -n kube-system -l svccontroller.k3s.cattle.io/svcname=web-lb -o name
curl -s http://$NODE:8081/api/actuator/health ; echo
kubectl delete svc web-lb
```

Observeer `web-lb LoadBalancer 10.43.x.x <IP-van-de-node> 8081:3xxxx/TCP` — het `EXTERNAL-IP` is het adres van de node —, een Pod `svclb-web-lb-…` in `kube-system`, en `{"status":"UP"}` op poort `8081`.

Uitleg. Kale Kubernetes zou `<pending>` laten; ServiceLB simuleert een load balancer door de poort op de node te openen. Poort 80 was al ingenomen door Traefik, vandaar `8081`.

Stap 5 — De Ingress

```
kubectl get svc -n kube-system traefik
kubectl apply -f ingress-web.yaml ; sleep 5 ; kubectl get ingress
curl -s http://localhost/ | head -3
curl -s http://localhost/api/actuator/health ; echo
curl -s http://$NODE/api/ ; echo
```

Observeer Traefik als `LoadBalancer` op `80:3xxxx/TCP,443:3xxxx/TCP` met het IP van de node; de Ingress `web` van klasse `traefik`, host `*`, `ADDRESS` = IP van de node; en dan de frontend en de API op **poort 80**, zonder NodePort: `http://localhost/`.

Uitleg. De Ingress beschrijft "`/` → Service `web`"; Traefik, dat al op poort 80 van de node zit, past het toe. Keten: poort 80 → Traefik → Service `web` → nginx → (`/api/`) Service `api` → Pod. Vanuit Windows werkt `http://localhost/` — k3s draait als root, de beperking op poorten < 1024 speelt niet.

Stap 6 — De endpoints volgen de Pods

```
POD=$(kubectl get pods -l app=api -o jsonpath='{.items[0].metadata.name}')
kubectl get endpointslices -l kubernetes.io/service-name=api -o
jsonpath='{.items[0].endpoints[*].addresses[0]}' ; echo
kubectl delete pod $POD ; sleep 1
kubectl get endpointslices -l kubernetes.io/service-name=api -o
jsonpath='{.items[0].endpoints[*].addresses[0]}' ; echo
kubectl port-forward svc/api 18080:8080 > /dev/null 2>&1 &
sleep 2 ; curl -s localhost:18080/actuator/health ; echo ; kill %1
```

Observeer drie adressen, en dan, één seconde na het verwijderen van een Pod, het adres van die Pod vervangen door dat van zijn opvolger (de ReplicaSet van lab 12 heeft hem opnieuw aangemaakt); en `port-forward` dat ook naar een **Service** werkt.

Uitleg. Het Service kent je Pods niet bij naam maar via hun labels en hun toestand `Ready`: de lijst wordt bij elke wijziging opnieuw berekend, zonder tussenkomst.

Opruimen

```
kubectl delete namespace lab13
kubectl config set-context --current --namespace=default
kubectl get svc -A | grep -c lab13
```

Observeer `0`: Services, Ingress en Deployments zijn met de namespace verdwenen. Hou `web:1.0` en `api:1.0` in Podman en in het cluster: de labs 14 en 15 vervolledigen de stack met de configuratie en de database.

Wat je nu moet kunnen beweren

- Zonder Service sterft nginx: `host not found in upstream "api"`; met het Service herstart hij vanzelf bij de volgende backoff.
- Een Service = `ClusterIP 10.43.x.x`, een naam geresolved door `10.43.0.10` via `search lab13.svc.cluster.local`, een EndpointSlice met de IP's van de klare Pods.
- `scale` naar 3 = drie endpoints; een verwijderde Pod verdwijnt uit de lijst en zijn vervanger komt erin.
- NodePort `30080` op het IP van de node; LoadBalancer = ServiceLB, `EXTERNAL-IP` = IP van de node, één Pod `svclb-…` per Service.
- Ingress `web` → Traefik op poort 80 van de node → `http://localhost/` en `http://localhost/api/actuator/health`.
- `port-forward svc/api` gaat door alles heen zonder iets bloot te stellen.